

WEEK 7 PRACTICAL

QUESTION 1

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Aim

To write a C program that displays the memory addresses of variables belonging to the **code**, **global**, **static**, **heap**, and **stack** segments and understand the Linux process address-space layout.

Program

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 100;

static int static_global_var = 200;

void sample_function() {
    printf("Inside sample function\n");
}

int main() {
    int stack_var = 300;

    static int static_local_var = 400;

    int *heap_var = (int *)malloc(sizeof(int));

    if (heap_var == NULL) {
        perror("malloc");
        return 1;
    }
    *heap_var = 500;
    printf("Memory Addresses:\n\n");

    printf("Code (function)           : %p\n", (void *)sample_function);
    printf("Global variable                : %p\n", (void *)&global_var);
    printf("Static global variable          : %p\n", (void *)&static_global_var);
    printf("Static local variable          : %p\n", (void *)&static_local_var);
    printf("Heap variable                   : %p\n", (void *)heap_var);
    printf("Stack variable                  : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

Output

The exact addresses will be different on every execution because of Linux memory-layout randomization.

Memory Addresses:

```
hrushika@Hrushikas-MacBook-Air week7 % gcc -Wall -Wextra q1_memory_layout.c -o memory_layout
./memory_layout
./memory_layout
Process ID: 8958
Memory Addresses:
-----
Code (sample_function)      : 0x100d78598
Code (main)                 : 0x100d785b4
Global (global_var, .data)  : 0x100d80000
Global (global_uninit_var,.bss) : 0x100d80010
Static (static_global_var)  : 0x100d80008
Static (static_uninit_var,.bss) : 0x100d8000c
Static (static_local_var)   : 0x100d80004
Heap (heap_var, malloc)     : 0x101551ba0
Stack (stack_var, main)     : 0x16f086b28
Stack (nested_call local)   : 0x16f086aec
-----

Observations to verify:
- Code has the lowest address, followed by .data/.bss, then the heap.
- The stack address is far higher than the heap address.
- The nested (deeper) stack frame has a LOWER address: the stack grows downward.
- Addresses change on every run because of ASLR.
Inside sample_function()
Process ID: 8961
Memory Addresses:
-----
Code (sample_function)      : 0x1045b4598
Code (main)                 : 0x1045b45b4
Global (global_var, .data)  : 0x1045bc000
Global (global_uninit_var,.bss) : 0x1045bc010
Static (static_global_var)  : 0x1045bc008
Static (static_uninit_var,.bss) : 0x1045bc00c
Static (static_local_var)   : 0x1045bc004
Heap (heap_var, malloc)     : 0x104c59ba0
Stack (stack_var, main)     : 0x16b84ab38
Stack (nested_call local)   : 0x16b84aafc
-----

Observations to verify:
- Code has the lowest address, followed by .data/.bss, then the heap.
- The stack address is far higher than the heap address.
- The nested (deeper) stack frame has a LOWER address: the stack grows downward.
- Addresses change on every run because of ASLR.
Inside sample_function()
hrushika@Hrushikas-MacBook-Air week7 %
```

Observation

The program displays the addresses of variables located in different parts of the process address space.

Memory Segment	Example	Purpose
Code / Text	sample_function	Contains executable program instructions
Global / Data	global_var	Stores initialized global variables
Static	static_global_var, static_local_var	Static variables have program-long lifetime and are typically stored in the data or BSS segment depending on initialization.
Heap	heap_var	Used for dynamically allocated memory using malloc()
Stack	stack_var	Stores local variables and function-call information

The addresses demonstrate that different types of data are maintained in different regions of the Linux process address space.

Result

The program was successfully implemented to print the addresses of **code, global, static, heap, and stack variables**. The Linux process memory layout was analyzed based on the addresses obtained.

QUESTION 2

Make use of the /proc/maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Aim

To study the memory organization of a running Linux process using the /proc/<PID>/maps file and memory-analysis commands, and understand virtual memory mappings.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;

int main() {
    int local_var = 200;

    printf("Process ID: %d\n", getpid());
    printf("Global variable address: %p\n", (void *)&global_var);
    printf("Stack variable address : %p\n", (void *)&local_var);

    printf("\nProcess is running...\n");
    printf("Open another terminal and execute:\n");
    printf("cat /proc/%d/maps\n", getpid());

    printf("\nPress Enter to terminate...\n");
    getchar();

    return 0;
}
```

Useful Memory Analysis Commands

To examine the process mappings:

```
cat /proc/1722/maps
```

Sample /proc/1722/maps Output

```
root@321434620fa9:/work# cat /proc/1070/maps
cat /proc/1070/status
head -40 /proc/1070/smaps
pmap 1070
aaaa4c0000-aaaa4c2000 r-xp 00000000 00:2c 7
aaaa4d0000-aaaa4e0000 r--p 0000f000 00:2c 7
aaaa4e0000-aaaa4e1000 rw-p 00010000 00:2c 7
aaaae21b0000-aaaae21df000 rw-p 00000000 00:00 0
ffff81400000-ffff81800000 rw-p 00000000 00:00 0
ffff8185f000-ffff81960000 rw-p 00000000 00:00 0
ffff81960000-ffff81afa000 r-xp 00000000 00:35 26290
ffff81afa000-ffff81b0d000 ---p 0019a000 00:35 26290
ffff81b0d000-ffff81b10000 r--p 0019d000 00:35 26290
ffff81b10000-ffff81b12000 rw-p 001a0000 00:35 26290
ffff81b12000-ffff81b1e000 rw-p 00000000 00:00 0
ffff81b2d000-ffff81b54000 r-xp 00000000 00:35 26270
ffff81b63000-ffff81b65000 rw-p 00000000 00:00 0
ffff81b65000-ffff81b69000 r--p 00000000 00:00 0
ffff81b69000-ffff81b6b000 r-xp 00000000 00:00 0
ffff81b6b000-ffff81b6d000 r--p 0002e000 00:35 26270
ffff81b6d000-ffff81b6f000 rw-p 00030000 00:35 26270
ffffd0590000-ffffd05b1000 rw-p 00000000 00:00 0
Name: proc_maps_demo
Umask: 0022
State: S (sleeping)
Tgid: 1070
Ngid: 0
Pid: 1070
PPid: 1
TracerPid: 0
Uid: 0 0 0
Gid: 0 0 0
FDSize: 256
Groups: 0
NSTgid: 1070
NSpid: 1070
NSpgid: 1070
NSsid: 1
```

To view memory statistics:

```
cat /proc/1722/status
```

To view detailed memory information:

```

root@321434620fa9:/work# head -40 /proc/1070/smaps
aaaaa4c0000-aaaaa4c2000 r-xp 00000000 00:2c 7
/work/proc_maps_demo
Size: 8 kB
KernelPageSize: 4 kB
MMUPageSize: 4 kB
Rss: 0 kB
Pss: 0 kB
Pss_Dirty: 0 kB
Shared_Clean: 0 kB
Shared_Dirty: 0 kB
Private_Clean: 0 kB
Private_Dirty: 0 kB
Referenced: 0 kB
Anonymous: 0 kB
KSM: 0 kB
LazyFree: 0 kB
AnonHugePages: 0 kB
ShmemPmdMapped: 0 kB
FilePmdMapped: 0 kB
Shared_Hugetlb: 0 kB
Private_Hugetlb: 0 kB
Swap: 0 kB
SwapPss: 0 kB
Locked: 0 kB
THPeligible: 0
VmFlags: rd ex mr mw me
aaaaa4df000-aaaaa4e0000 r--p 0000f000 00:2c 7
/work/proc_maps_demo
Size: 4 kB
KernelPageSize: 4 kB
MMUPageSize: 4 kB
Rss: 4 kB
Pss: 4 kB
Pss_Dirty: 4 kB
Shared_Clean: 0 kB
Shared_Dirty: 0 kB
Private_Clean: 0 kB
Private_Dirty: 4 kB
Referenced: 4 kB
Anonymous: 4 kB
KSM: 0 kB
LazyFree: 0 kB
root@321434620fa9:/work#

HugetlbPages: 0 kB
CoreDumping: 0
THP_enabled: 1
untag_mask: 0xffffffffffffff
Threads: 1
SigQ: 0/31879

```

cat /proc/1722/smaps

You can also use:

pmap 1722

Observation

```

LazyFree: 0 KB
root@321434620fa9:/work# pmap 1070
1070:  ./proc_maps_demo 300
0000aaaada4c0000      8K r-x-- proc_maps_demo
0000aaaada4df000      4K r---- proc_maps_demo
0000aaaada4e0000      4K rw--- proc_maps_demo
0000aaaae21be000     132K rw--- [ anon ]
0000ffff81400000    4096K rw--- [ anon ]
0000ffff8185f000    1028K rw--- [ anon ]
0000ffff81960000    1640K r-x-- libc.so.6
0000ffff81afa000      76K ----- libc.so.6
0000ffff81b0d000      12K r---- libc.so.6
0000ffff81b10000       8K rw--- libc.so.6
0000ffff81b12000      48K rw--- [ anon ]
0000ffff81b2d000     156K r-x-- ld-linux-aarch64.so.1
0000ffff81b63000       8K rw--- [ anon ]
0000ffff81b65000      16K r---- [ anon ]
0000ffff81b69000       8K r-x-- [ anon ]
0000ffff81b6b000       8K r---- ld-linux-aarch64.so.1
0000ffff81b6d000       8K rw--- ld-linux-aarch64.so.1
0000ffffd0590000     132K rw--- [ stack ]
total                7392K
root@321434620fa9:/work#

```

The `/proc/<PID>/maps` file shows the **virtual memory regions** assigned to a running process.

Common mappings include:

- **Executable/text region** — contains the program's machine instructions.
- **Read-only region** — contains read-only data and other protected mappings.
- **Data region** — contains initialized and writable program data.
- **Heap** — used for dynamically allocated memory.
- **Shared libraries** — libraries such as the C library are mapped into the process's virtual address space.
- **Stack** — contains local variables and function-call information.
- **VDSO/VVAR** — special kernel-provided mappings used for certain system operations and timing-related functionality.

The permissions shown in `/proc/<PID>/maps` describe what operations are allowed:

r = read

w = write

x = execute

p = private mapping

s = shared mapping

Result

The memory organization of a running Linux process was successfully studied using `/proc/<PID>/maps`, `/proc/<PID>/status`, `/proc/<PID>/smaps`, and `pmap`. The experiment demonstrated how **virtual memory mappings** divide a process's address space into code, data, heap, stack, libraries, and other regions.